

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



Department of Computer Science & Engineering

24 Series

Course Code: CSE 1201

Course Title: Data Structure

Assignment No: 03

Assignment Title: Investigation of Algorithm Efficiency in Real-Life
Data Processing Systems

Submitted by:

Fardin Bin Aslam Numan

Roll: 2403179

Section: C

Session: 2024-25

Submitted to:

Emrana Kabir Hashi

Assistant Professor

Dept. of Computer Science &
Engineering, RUET

Date of Submission: 03/02/2026

Online Food Delivery Order Management System

Introduction

In modern real life data processing systems, algorithm efficiency plays a critical role in ensuring fast response time, scalability and optimal resource utilization. Online food delivery platforms like *foodpanda* or *foodi* handle a large volume of customer orders daily, each containing attributes such as order ID order amount, delivery distance and order time. Efficient handling of this data is critical for billing accuracy, delivery optimization, customer support and overall system responsiveness. To achieve acceptable performance at scale, such systems must rely on appropriate algorithms whose efficiency is well understood under different real world data conditions

Task 1: Data Characterization

To simulate real world conditions of an online food delivery system, three dataset sizes are considered...

- **Small dataset (≤ 100 orders):** Typically observed in newly launched platforms or during off-peak hours
- **Medium dataset (≈ 1000 orders):** Represents regular daily operations in a moderately active system
- **Large dataset (≥ 10000 orders):** Reflects peak-hour usage or long term accumulated order data

Additionally order data can appear in different arrangements based on system usage patterns...

- **Random data:** Orders are stored in no particular order, common in real time order placement which arrive unpredictably throughout the day
- **Nearly sorted data:** Orders are mostly sorted by time or order ID, with a few recent insertions
- **Reverse sorted data:** Occurs when newer orders are placed before older ones in storage

Understanding these data characteristics is crucial because algorithm performance varies significantly depending on data order and size

Task 2: Sorting Algorithm Investigation

2.1 | Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order

- **Time Complexity**
 - Best case: $O(n)$ - when data nearly sorted
 - Average case: $O(n^2)$ - due to nested loops
 - Worst case: $O(n^2)$ - due to nested loops
- **Suitability**
 - Suitable only for very small datasets (≤ 100)
 - Performs reasonably on nearly sorted data but poorly on large or reverse-sorted data
- **Practical Limitations**
 - Excessive comparisons and swaps
 - Not scalable for real systems
 - Impractical for medium/large data due to quadratic time
- **Space Complexity**
 - $O(1)$, as it is an in-place algorithm

2.2 | Insertion Sort

Insertion Sort builds the sorted list one element at a time by placing elements in their correct position

- **Time Complexity**
 - Best case: $O(n)$ - nearly sorted data
 - Average case: $O(n^2)$ - random sorted
 - Worst case: $O(n^2)$ - reverse sorted
- **Suitability**
 - Highly effective for small and nearly sorted datasets
 - Inefficient for large, unsorted data
- **Practical Limitations**
 - High execution time and poor scalability for large datasets
 - Slow for large random data
- **Space Complexity**
 - $O(1)$, in-place

2.3 | Selection Sort

Selection Sort repeatedly selects the minimum element and places it at the correct position

- **Time Complexity**
 - Best case: $O(n^2)$ - always scans unsorted portion fully
 - Average case: $O(n^2)$ - always scans unsorted portion fully
 - Worst case: $O(n^2)$ - always scans unsorted portion fully
- **Suitability**
 - Acceptable only for very small datasets
- **Practical Limitations**
 - Fixed number of comparisons regardless of data order
 - Even best case is quadratic.
- **Space Complexity**
 - $O(1)$, in-place

2.4 | Quick Sort

Quick Sort uses a divide-and-conquer strategy by selecting a pivot and partitioning the array

- **Time Complexity**
 - Best case: $O(n \log n)$ - good pivot choice
 - Average case: $O(n \log n)$ - good pivot choice
 - Worst case: $O(n^2)$ - when pivot is smallest/largest repeatedly (sorted/reverse data)
- **Suitability**
 - Very efficient for medium to large datasets
 - Performs well on random data
- **Practical Limitations**
 - Poor pivot selection can degrade performance
 - Worst case risky without randomization; recursive stack $O(\log n)$ avg, $O(n)$ worst.
- **Space Complexity**
 - $O(\log n)$ (recursive stack)

2.5 | Merge Sort

Merge Sort divides the dataset into halves, sorts them recursively and merges the results

- **Time Complexity**
 - Best case: $O(n \log n)$ - consistently divides and merges
 - Average case: $O(n \log n)$ - consistently divides and merges
 - Worst case: $O(n \log n)$ - consistently divides and merges
- **Suitability**
 - Reliable for large datasets, especially when stability is required
 - Predictable and consistent performance regardless of data order
- **Practical Limitations**
 - Requires additional memory
 - $O(n)$ extra space; slower constant factors than Quick Sort for random data.
- **Space Complexity**
 - $O(n)$ auxiliary

Task 3: Searching Algorithm Investigation

3.1 | Linear Search

Linear Search checks each order sequentially until the target is found.

- **Performance:**
 - Works on both sorted and unsorted data
 - Acceptable for small datasets or infrequent searches
- **Usage Scenario:**
 - Suitable for small or unsorted datasets

- **Time Complexity:**
 - Best case: $O(1)$
 - Average case: $O(n)$ - for both sorted/unordered
 - Worst case: $O(n)$ - for both sorted/unordered
- **Limitation:**
 - Becomes inefficient as dataset size increases

3.2 | Binary Search

Binary Search repeatedly divides the search space in half.

- **Performance:**
 - Extremely fast on large datasets
 - Requires pre-sorted data
- **Usage Scenario**
 - Requires sorted data
 - Ideal for large datasets
- **Time Complexity**
 - Best case: $O(1)$
 - Average and Worst case: $O(\log n)$ - requires sorted array
- **Trade-off:**
 - Sorting overhead required before searching

Preprocessing Justification:

For one time search (locating a single order ID during a rare complaint case), linear search is better. For frequent search operations (customer support queries), sorting once and applying binary search significantly improves overall system performance. Although sorting introduces an initial cost, it reduces repeated search time in long term usage

Task 4: Comparative Analysis

4.1 | Comparison Table

Dataset Size	Algorithm Used	Complexity		Why Recommended	Expected Performance
		Time	Space		
Small (Random)	Insertion Sort	$O(n^2)$	$O(1)$	Low overhead; performs well for small input sizes	Fast and simple
Small (Nearly Sorted)	Insertion Sort	$O(n)$	$O(1)$	Best for nearly sorted; efficient	Very fast
Medium (Random)	Quick Sort (randomized)	$O(n \log n)$	$O(\log n)$	Efficient average case performance with in-place sorting	Efficient

Medium (Nearly Sorted)	Insertion / Quick Sort	$O(n)$ / $O(n \log n)$	$O(1)$ / $O(\log n)$	Insertion excels if disorder is minimal; otherwise Quick Sort remains effective	Efficient
Large (Random)	Merge Sort	$O(n \log n)$	$O(n)$	Predictable performance; suitable for large-scale systems	Stable and reliable
Large (Reverse Sorted)	Merge Sort	$O(n \log n)$	$O(n)$	Avoids Quick Sort's worst case degradation	Stable and reliable
Searching (Sorted)	Binary Search	$O(\log n)$	$O(1)$	Extremely fast lookup after preprocessing	Very fast
Searching (Unsorted)	Linear Search	$O(n)$	$O(1)$	No sorting overhead for infrequent or one- time searches	Slow for large data

4.2 | Scalability Analysis

Why some algorithms become impractical as data grows?

Algorithms with quadratic complexity (Bubble, Selection, Insertion) become impractical as dataset size grows due to exponential increase in execution time (Scaling from 100 to 10000 orders increases operations $\sim 10000\times$, causing delays). In contrast, $O(n \log n)$ algorithms scale efficiently even for large datasets

4.3 | Impact of Poor Algorithm Selection

- **Response time:** Slows from milliseconds to minutes
- **Resource consumption:** High CPU usage, memory thrashing
- **User experience:** Laggy UI, timeout errors, poor satisfaction

Task 5: Recommendation & Decision Making

Based on the investigation of dataset characteristics, algorithm complexity, and practical requirements of an online food delivery system, the following recommendations are made...

5.1 | Daily Order Processing:

Recommended Sorting Algorithm: Merge Sort

Justification:

Daily order datasets often exceed 10000 entries across multiple restaurants. Merge Sort provides stable $O(n \log n)$ performance, independent of data order (random, nearly sorted or reverse sorted), ensuring predictable processing time for billing, analytics and delivery

planning. For example, sorting 10000 daily orders by delivery distance using Merge Sort requires approximately 130000 comparisons ($O(n \log n)$), whereas Bubble Sort would require roughly 50 million comparisons ($O(n^2)$), making Merge Sort vastly faster and more predictable in real time processing

Recommended Searching Algorithm: Binary Search

Justification:

Once the dataset is sorted by order ID or timestamp, Binary Search enables $O(\log n)$ lookup, allowing rapid retrieval of specific orders for reporting, customer support or issue resolution. For example, looking up order #179 out of 10000 orders can be done in ≤ 14 comparisons, compared to 10000 comparisons using Linear Search

5.2 | Real-Time Customer Query Handling:

Recommended Sorting Algorithm: Insertion Sort

Justification:

Real time queries usually involve small, recent batches of orders (≤ 100 -200 orders), which are often nearly sorted by timestamp. Insertion Sort adapts efficiently to these conditions, introducing minimal overhead while maintaining sorted order for rapid search. For example, a delivery agent or customer service agent needs to retrieve the last 50 orders of a restaurant. Using Insertion Sort keeps this small dataset ordered for quick searches without the cost of Merge Sort

Recommended Searching Algorithm: Binary Search

Justification:

Sorting the subset allows fast $O(\log n)$ searches, critical for real time queries where response time affects customer satisfaction. For example, finding order #79 in a subset of 100 recent orders using Binary Search requires only ~ 7 comparisons ($\log_2 100 \approx 6.64$), compared to 100 comparisons if Linear Search were used

Conclusion

This investigation demonstrates that algorithm efficiency plays a critical role in real life data processing systems. While simple algorithms may perform adequately for small datasets, scalable systems demand algorithms with logarithmic or linearithmic complexity. As a result, selecting appropriate sorting and searching techniques directly impacts system responsiveness, resource utilization and user satisfaction

Reflection

Algorithm complexity has a direct influence on the scalability of real life systems. As data volume increases, inefficient algorithms that appear acceptable at small scales quickly become performance bottlenecks. For example, an $O(n^2)$ sorting algorithm that works fine for 100 orders will fail catastrophically at 10000 orders, causing exponential resource growth. This

investigation highlights that understanding time and space complexity is not merely theoretical but essential for designing systems capable of handling growth

Although worst case analysis provides a safety bound, average case analysis is often more meaningful in practice because real world data rarely represents worst case scenarios consistently. For example, algorithms like Quick Sort perform exceptionally well on average, making them suitable for most operational environments despite unfavorable worst case possibilities. Using inefficient algorithms in large systems introduces significant risks, including increased response times and operational costs, excessive resource consumption and poor user experience. In customer-facing platforms such as food delivery systems, even minor delays can impact customer trust and business revenue

This assignment significantly improved my understanding that algorithm selection is a strategic decision rather than a purely academic concept. It emphasized that choosing the right algorithm is not only about correctness but also about efficiency, scalability and real world applicability. I now understand why companies like *foodpanda* or *foodi* rely on hybrid algorithms because real data is often partially sorted and adaptive algorithms exploit that. As a result, I now approach algorithm design with a stronger focus on data characteristics, performance trade-offs and long term system behavior